

Security Architecture & Threat Model – SecOpsAI

Project 3 - Team ThreatMatrix - Expadox Lab

Repository: github.com/rootjoker311/secops

1. System Context

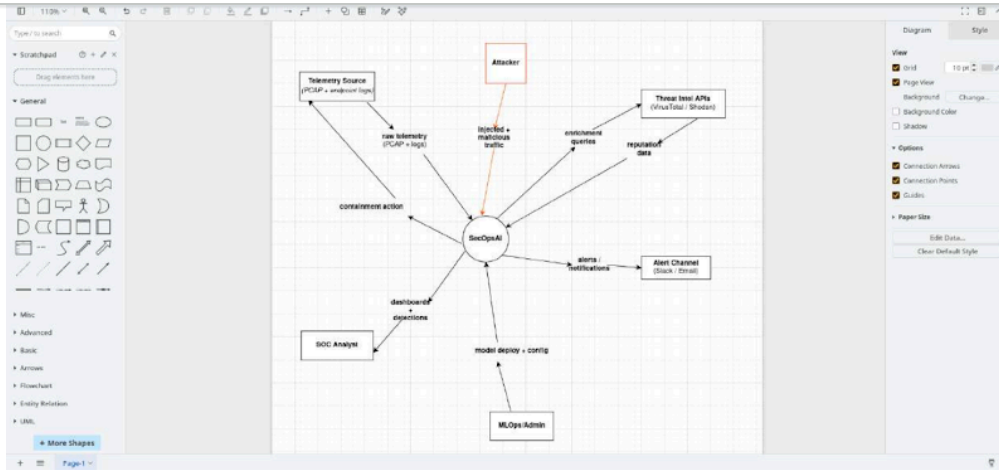
SecOpsAI is an AI-driven detection layer that sits inside the SOC & augments the SIEM catching behavioral threats that signature-based IDS/IPS cannot.

The system ingests network and endpoint telemetry, scores it with a machine-learning detections with external threat intelligence, and drives an automated alert-and-response pipeline.

At the highest level, the platform interacts with six external actors:

- **Telemetry Source** — provides the raw network captures (PCAP) and endpoint logs the system analyses.
- **Attacker** — an untrusted actor who both generates malicious traffic and attempts to deceive the model itself.
- **SOC Analyst** — consumes dashboards, detections, and alerts.
- **MLOps / Admin** — deploys models and manages configuration.
- **Threat Intel APIs** (VirusTotal / Shodan) — provide reputation and enrichment data.
- **Alert Channel** (Slack / Email) — receives outbound notifications.

← Security Architecture & Threat Model – SecOpsAI



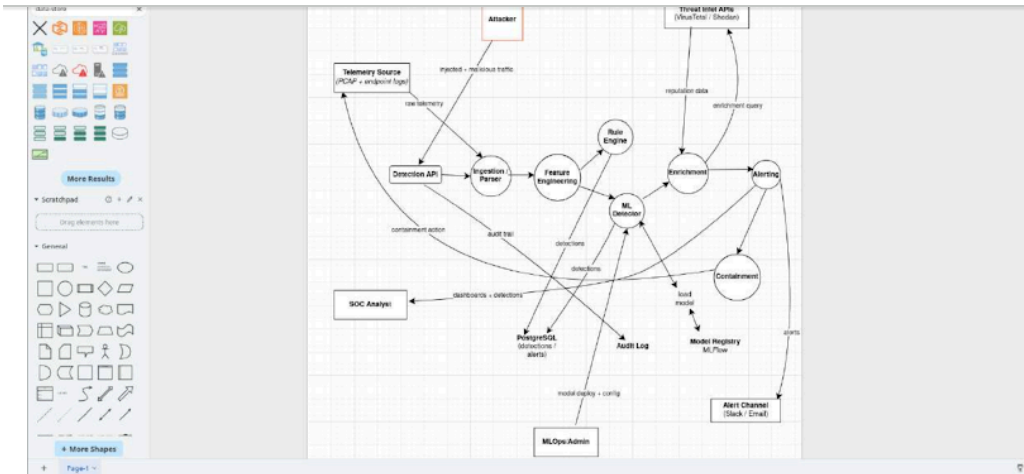
2. Data Flow Diagram & Trust Boundaries

Opening up the system, telemetry flows through a pipeline of internal processes — ingestion, feature engineering, a rule-based engine and an ML detector running in parallel, then enrichment, alerting, and containment — backed by a PostgreSQL store, an audit log, and an MLflow model registry. We defined four trust boundaries: points where data crosses from a less-trusted zone into a more-trusted one. Each boundary is where an attacker is most likely to strike, so each becomes a focus of the threat analysis.

ID	Trust Boundary	What it separates
TB1	API Edge	Attackers ↔ Detection API
TB2	Detection Core	API and ingestion ↔ ML model and data stores
TB3	Response Zone	Detection logic ↔ automated containment actions
TB4	Egress Boundary	Our system ↔ third parties services (VirusTotal, Shodan, Slack)

A **CI/CD supply-chain boundary** protects how code and models reach to production.

← Security Architecture & Threat Model – SecOpsAI



3. STRIDE Threat Model

For each trust boundary we applied the six STRIDE categories — Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege.

As we're working on AI system so we've extended the analysis on the ML model with three AI-specific threats:

Poisoning, evasion & inference.

Classic threats are mapped to **MITRE ATT&CK**. The AI-specific threats are mapped to **MITRE ATLAS**, the adversarial-machine-learning framework that extends ATT&CK.

TB1 — API Edge (Detection API)

The Detection API is the system's public-facing entry point, so it is the first surface any attacker interacts with.

STRIDE	Threat	Mitigation	MITRE
--------	--------	------------	-------

Spoofing	An attacker pretends to be a real user or uses a stolen token to access the API.	Requires login with API key/JWT; expire tokens quickly.	T1078
Tampering	A broken or oversized request is sent to crash or confuse the API.	Validate every input; limit request size; reject unknown data.	T1565

Info Disclosure	Error messages leaks internal or model details.	Show generic errors; use TLS; hide internals.	T1190
Denial of Service	The API are flooded so it can't detect real threats.	Rate limiting, Queue and timeouts.	T1499
Elevation of Privilege	A vulnerability is exploited to break into the internal system & elevate the privilege to admin or root user	Run with least privilege; harden containers; scan dependencies.	T1068

TB2 — Detection Core (ML Model)

The most important part of the threat model cause the first three rows are conventional threats against the model and it's data, the final three are the AI-specific threats that define an adversarial-ML system.

STRIDE/AI	Threat	Mitigation	MITRE
-----------	--------	------------	-------

Tampering	The trained model files or its weights are altered in storage so the detector behaves incorrectly.	Signed, versioned models in MLflow, integrity hashes, and strict access control on the registry.	ATT&CK T1565
Info Disclosure	Training datas or engineered features leaks to unauthorised parties.	Encryption at rest and tight access control on the data store and registry.	ATT&CK T1530
Denial of Service	A deliberately heavy or crafted input spikes inference latency and breaks the p99 < 200ms requirement.	Input size caps, inference timeouts, and a lightweight model (XGBoost).	ATT&CK T1499
Poisoning(AI)	Because some training data originates from live traffic, an attacker can inject crafted payloads to corrupt the original data over time so a future retrained model learns to treat their attacks as normal.	Train the baseline only on the trusted, version-controlled CICIDS2017 dataset; treat live data as untrusted and quarantine it for human review before any retraining; tamper-evident logging and integrity checks on the	ATLAS AML.T0020

	traffic that remains malicious but is tweaked just enough to cross the model's decision boundary and be scored as benign — for example, adding jitter to C2 beaconing so its periodicity looks irregular.	using IBM ART, feature robustness, an ensemble of the rule engine and the ML model so both must be fooled, and input anomaly detection.	
Inference/Extraction(AI)	An attacker repeatedly probes the API to learn the model's decision logic or to reconstruct an equivalent copy of the model.	Rate limiting on predictions, withholding confidence scores from responses, and monitoring for abnormal query patterns.	ATLAS AML.T0024 / T0040

TB3 — Response Zone (Enrichment · Alerting · Containment)

This zone is where the system takes action — enriching detections, raising alerts, and executing automated containment. Because these actions have real-world effect, the threats here focuses on misuse of that automation.

STRIDE	Threat	Mitigation	MITRE
Spoofing	A forged detection event is injected to trick the system into containing something it should not.	Authenticate the internal event source and sign event messages.	ATT&CK T1036
Tampering	An alert or containment command is altered while in transit between services.	Integrity-checked messages on Redis Streams and TLS between components.	ATT&CK T1565
Repudiation	A containment action occurs with no record of which detection triggered it.	Log every response action together with the ID of the detection that caused it.	ATT&CK T1070
Denial of Service	An attacker deliberately triggers many false positives so the system auto-contains legitimate	Requires a confidence threshold or rate-cap blocking actions or queue limiting	ATT&CK T1499

	weapon.		
Elevation of Privilege	The containment service holds excessive control over the network and is hijacked to cause wider harm.	Least-privilege firewall scope and, in the lab, a mocked or isolated containment action.	ATT&CK T1068

TB4 — Egress Boundary (Third-Party Services)

This trust boundary covers how the data leaves from our system to external third party services. The concern is that what to disclose and whether we can trust what comes back to our system whether it is spoofed or not.

STRIDE	Threat	Mitigation	MITRE
--------	--------	------------	-------

Information Disclosure	Sensitive internal IOCs are sent to 3rd party platforms like VirusTotal, Shodan & Any.RUN	Send only the minimum necessary indicators, scope API keys tightly, and also review what is being shared to the platforms.	ATT&CK T1567
Spoofing	A forged threat-intelligence response feeds the enrichment process bad data.	Validate API responses and pin third-party endpoints and certificates.	ATT&CK T1036
Denial of Service	A third-party rate limit, queue system or outage stalls the enrichment pipeline.	Cache results and degrade gracefully when intelligence is unavailable.	ATT&CK T1499

CI/CD Supply Chain

This trust boundary protects the pathway from which the code and models reaches production.

STRIDE	Threat	Mitigation	MITRE
--------	--------	------------	-------

	production through the pipeline	committing/merging a regression test should be performed if passed then it should be pushed into production	
Elevation of Privileges	Leaked CI/CD secrets/credentials can impact which can cause an attacker to gain access to production	Secret scanning and scoped, least privilege tokens for pipeline runners	ATT&CK T1078

4. Architecture Decision Record(ADR)

The ADR records the key design choices and, importantly, what was deliberately left out and why.

Decision	Tool Use/Tool Details	Rationale	Trade-off/Left
----------	-----------------------	-----------	----------------

XGBoost as primary model	Gradient-boosting algorithm that builds many small decision trees in sequence, each fixing the last one's errors; excellent on tabular data.	Strong F1 with very fast inference (meets p99 < 200ms) and built-in feature importance for the ablation study.	Deep learning left out — overkill for tabular flow data and slower to serve.
CICIDS2017 dataset	A labeled network-traffic dataset containing benign + attack flows (DoS, brute force, infiltration, etc.).	Well-labeled and covers the required attack behaviors with minimal cleaning.	Single dataset — no cross-dataset generalisation tested this week.
Redis Streams for transport	In-memory data store with a lightweight streaming/message-queue mode.	Far simpler to operate than Kafka within a one-week build, while still providing pipeline streaming.	Less throughput/durability than Kafka; realised as a stand-in this week.
WAZUH for SIEM	Open-source SIEM + endpoint-monitoring platform; the SOC integration point for alerts.	Faster, lower-risk integration since the team already used it in Project 1.	Integration designed but not wired within the cohort week.
Suricata as rule baseline IDS/IPS	High-performance signature-based IDS/IPS.	It is the "legacy" detection the project aims to beat, so it's the natural baseline.	Operationalised as an SMD threshold panel for a like-for-like metric (see §4).

← Security Architecture & Threat Model – SecOpsAI

	helps harden them.	robustness (supports the $\geq 3/6$ survive bar).	
FastAPI for the API	Modern async Python web framework with automatic OpenAPI docs.	Fast, generates the required OpenAPI spec, built-in Pydantic validation supports security-by-design.	No built-in auth/rate-limit — implemented manually in <code>serve_api.py</code>
MLflow for model ops	Experiment-tracking platform with a model registry.	Provides evidence for the F1 comparison and signed model versions (mitigates model tampering).	Replaced by versioned JSON artefacts with hash-chain integrity this week.

5. Tool Stack

Layer	Tool	Role
Language / runtime	Python 3.13	Whole system, run from a venv on Kali
Model	XGBoost	Behavioural detector (gradient-boosted trees)
Adversarial testing	IBM ART	Gradient-free evasion attacks + hardening
Explainability	SHAP	Per-alert feature attribution for analysts
API	FastAPI + Uvicorn	Secured scoring service (OpenAPI, async)
Validation	Pydantic	Schema/type validation at the API edge
Testing / coverage	pytest + pytest-cov	Unit, security, and regression tests
Statistics	statsmodels, NumPy	McNemar test, bootstrap confidence interval
Data	pandas, PyArrow	CICIDS2017 ingestion, parquet feature store
Containment	nftables	Firewall block rules in the response pipeline
CI/CD	GitHub Actions	Detection-as-code regression gate on every push
Dataset	CICIDS2017	Labelled benign + attack network flows

6. Implementation Scope (Cohort Week)

Section 2 is the **target architecture**; within one week several components were realised as functionally equivalent stand-ins, with the full stack as the roadmap:

- **Rule baseline** → an SMD-calibrated feature-threshold panel on the CICIDS2017 split, not Suricata. Suricata scores raw PCAP while the model scores flow-feature vectors, so they cannot share one test set and metric; the panel is the *measurable signature analog*, Suricata the conceptual layer.
- **Model registry** → versioned JSON artefacts (`xgb_model.json`, `xgb_hardened.json`) with hash-chain integrity; MLflow is the production target.
- **Transport / SIEM / datastore** → self-contained module, no Wazuh wiring, parquet/JSON in place of Redis Streams and PostgreSQL.
- **Threat-intel enrichment** → local stub; VirusTotal / Shodan are the production integrations.

7. Data Pipeline & Feature Engineering

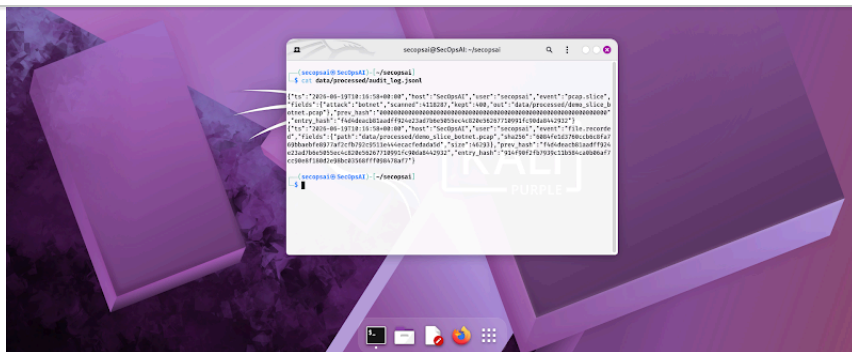
CICIDS2017 was ingested from eight CSVs (**2,827,876 flows**). CICFlowMeter emits infinite rate values on zero-duration flows (a divide-by-zero artefact) that silently corrupt training, so infinities are converted to missing values and 2,867 unrecoverable rows dropped. Eighteen features were retained, including **jitter-resistant features** (IAT autocorrelation, coefficient of variation, FFT dominant power, packet-size entropy, small-packet ratio) engineered against the ATLAS AML.T0015 evasion threat: raw timing is easy to disguise, but a beaconing channel's higher-order structure is not.

Feature value was quantified by standardized mean difference (SMD), reused later to build the baseline:

```
def standardized_mean_diff(x, y):
    a, b = x[y == 1], x[y == 0]      # attack vs benign
    pooled = np.sqrt((np.nanvar(a) + np.nanvar(b)) / 2.0)
    if pooled == 0 or np.isnan(pooled):
        return 0.0
    return float((np.nanmean(a) - np.nanmean(b)) / pooled)
```

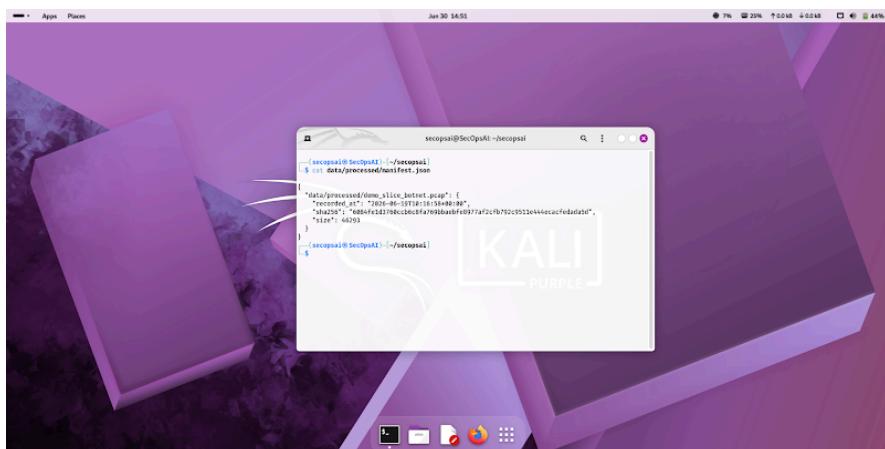
Integrity is protected by a hash-chained audit log with a SHA-256 manifest (the Tampering/Repudiation mitigation, and integrity stand-in for MLflow): each record's hash incorporates the previous one, so any change breaks the chain. A four-case adversarial injection test passed 4/4.

← Security Architecture & Threat Model – SecOpsAI



audit.json

manifest.json



8. Behavioural Machine Learning

← Security Architecture & Threat Model – SecOpsAI

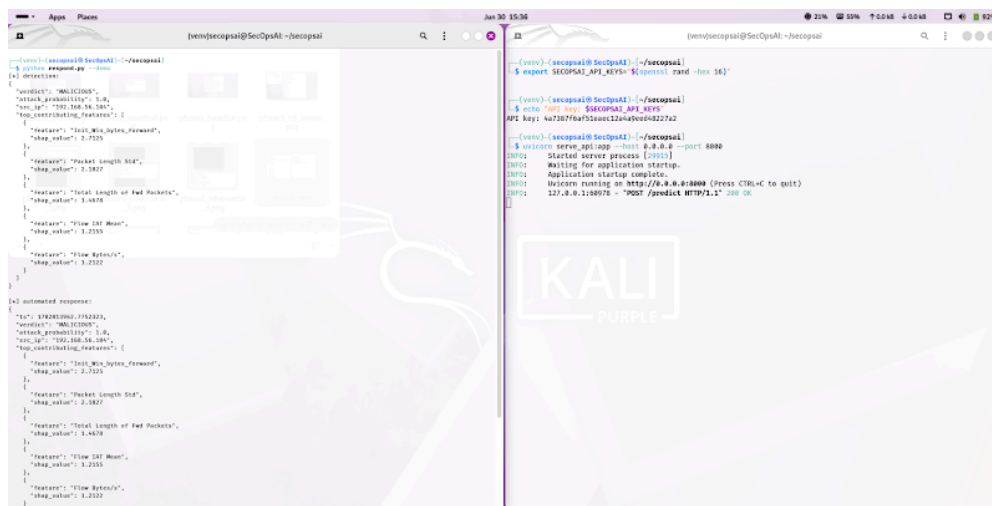
0.5635, precision 0.4725 — meaning more than half of every alert it raises is a false positive. That alert-fatigue failure mode is exactly what the model must fix.

Model. XGBoost on the identical split reaches **F1 0.9967** (precision 0.9940, recall 0.9994; TP 166,864 / FP 1,002 / FN 103 / TN 680,394). The false-positive rate collapses from ~52% to under 1% while recall approaches total — gradient boosting captures feature interactions no single threshold can.

Proof. The **+0.4332** uplift is defended two ways on the same test set:

```
# McNemar: do the two detectors' error patterns genuinely differ?
res = mcnemar([[both_correct, base_right_xgb_wrong],
              [base_wrong_xgb_right, both_wrong]], correction=True)
# Bootstrap: resample the test set 1000x; is the F1 gap ever near zero?
for i in range(n_boot):
    s = M[:, rng.integers(0, n, n)].sum(axis=1) # TP/FP/FN counts per resample
    diffs[i] = f1(s[0], s[1], s[2]) - f1(s[3], s[4], s[5])
```

McNemar returns $\chi^2 \approx 178,046$, $p \approx 0$ (180,107 baseline errors fixed, 690 introduced); the bootstrap CI is **[+0.4314, +0.4350]**, never near zero. Every malicious verdict ships with a SHAP top-five (e.g. Init_Win_bytes_forward, Packet Length Std, Total Backward Packets, Total length of Fwd Packets, Flow IAT Mean) so the analyst sees why a flow was flagged.



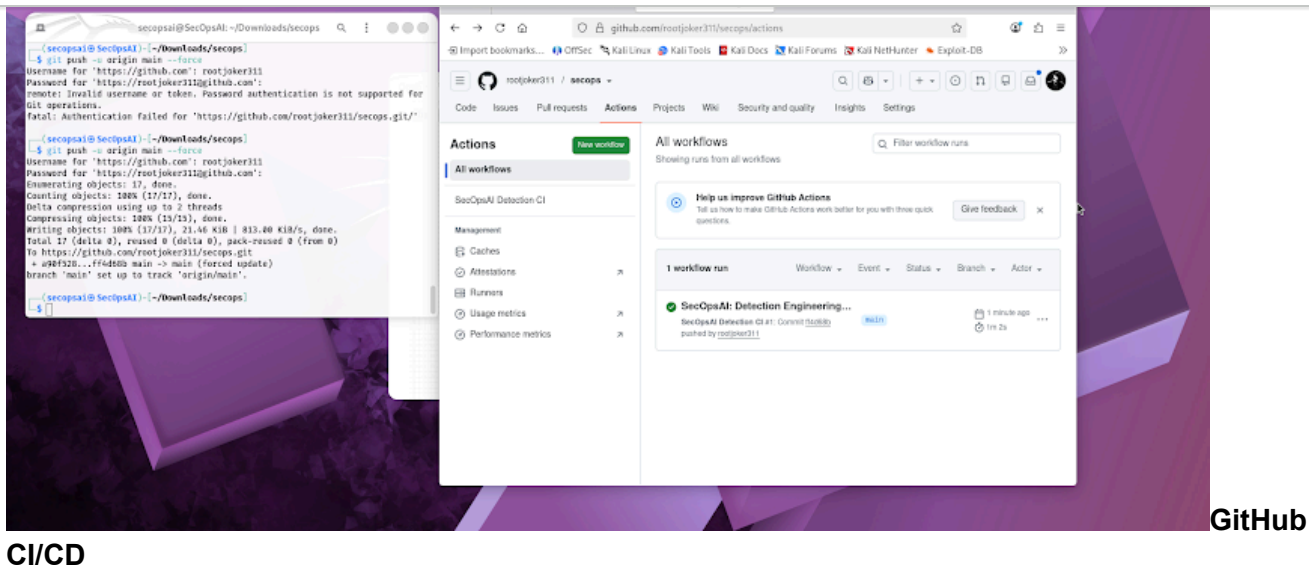
[respond.py](#) –demo

```
(venv)-(secopsai@SecOpsAI)-[~/secopsai]
$ pytest test_regression.py
===== test session starts =====
platform linux -- Python 3.13.12, pytest-9.1.0, pluggy-1.6.0
rootdir: /home/secopsai/secopsai
plugins: cov-7.1.0, anyio-4.13.0
collected 4 items

test_regression.py .... [100%]
===== 4 passed in 0.05s =====
```

test_regression.py

← Security Architecture & Threat Model – SecOpsAI



CI/CD

9. Adversarial Hardening

XGBoost has no gradients, so gradient attacks (FGSM/PGD) do not apply — the panel uses gradient-free methods: three ART black-box attacks (ZOO, HopSkipJump, Boundary) and three constrained, realistic perturbations that only touch attacker-controllable features and directly test the jitter-resistant features:

```
def perturb(X, cols_idx, lo, hi, eps, mode, rng):
    sub = X[:, cols_idx]
    if mode == "jitter": sub = sub * (1 + rng.uniform(-eps, eps, sub.shape)) # timing
    elif mode == "pad": sub = sub + eps*(hi-lo)[cols_idx]*rng.uniform(0,1,sub.shape) # size
    elif mode == "slow": sub = sub * (1 - rng.uniform(0, eps, sub.shape)) # rate
    X[:, cols_idx] = np.clip(sub, lo[cols_idx], hi[cols_idx]); return X
```

Finding. Pre-hardening the model survived 5/6 — but **packet padding evaded at 100%**. This ties to the SHAP analysis: the model leaned on size features, so inflating packet sizes attacks exactly the signal it trusted most. *The property that made it accurate made it evadable.*

Fix. Adversarial training folds the successful evasions back into training and retrain. Survival rises to **6/6** (padding 100% → 0%), and clean-test F1 holds at **0.9966** — robustness gained without trading accuracy. Caveat: the padding fix partly trains for the tested attack, but the three ART attacks (not in the augmentation set) also held at/near 0%.

← Security Architecture & Threat Model – SecOpsAI

```
(venv)-(secopsai@SecOpsAI)-[~/secopsai]
$ python adversarial_hardening.py
[*] attacker-controllable feature groups:
  timing: ['Flow IAT Mean', 'Flow IAT Std', 'Flow IAT Min', 'Flow IAT Max', 'Active Mean', 'Idle Mean', 'Flow Duration']
  size: ['Packet Length Std', 'Total Length of Fwd Packets', 'Total Backward Packets', 'Fwd Packet Length Max', 'Init_Win_bytes_forward']
  rate: ['Flow Bytes/s', 'Flow Packets/s']

[*] evaluating on 150 currently-detected attack flows

[*] PRE-HARDENING panel:
[1/6] ZOO ...
ZOO: 100% | 1/1 [00:00:00:00, 25.35it/s]
ZOO: 100% | 1/1 [00:00:00:00, 40.37it/s]
ZOO: 100% | 1/1 [00:00:00:00, 138.79it/s]
ZOO: 100% | 1/1 [00:00:00:00, 33.35it/s]
ZOO: 100% | 1/1 [00:00:00:00, 36.22it/s]
ZOO: 100% | 1/1 [00:00:00:00, 16.41it/s]
ZOO: 100% | 1/1 [00:00:00:00, 24.15it/s]
ZOO: 100% | 1/1 [00:00:00:00, 85.10it/s]
ZOO: 100% | 1/1 [00:00:00:00, 29.83it/s]
ZOO: 100% | 1/1 [00:00:00:00, 125.41it/s]
ZOO: 100% | 1/1 [00:00:00:00, 40.78it/s]
ZOO: 100% | 1/1 [00:00:00:00, 149.67it/s]
ZOO: 100% | 1/1 [00:00:00:00, 123.41it/s]
ZOO: 100% | 1/1 [00:00:00:00, 41.63it/s]
ZOO: 100% | 1/1 [00:00:00:00, 140.91it/s]
ZOO: 100% | 1/1 [00:00:00:00, 30.94it/s]
ZOO: 100% | 1/1 [00:00:00:00, 145.71it/s]
ZOO: 100% | 1/1 [00:00:00:00, 15.57it/s]
ZOO: 100% | 1/1 [00:00:00:00, 60.42it/s]
ZOO: 100% | 1/1 [00:00:00:00, 21.43it/s]
ZOO: 100% | 1/1 [00:00:00:00, 41.80it/s]
ZOO: 100% | 1/1 [00:00:00:00, 20.06it/s]
ZOO: 100% | 1/1 [00:00:00:00, 145.16it/s]
ZOO: 100% | 1/1 [00:00:00:00, 38.32it/s]
ZOO: 100% | 1/1 [00:00:00:00, 65.41it/s]
ZOO: 100% | 1/1 [00:00:00:00, 30.65it/s]
ZOO: 100% | 1/1 [00:00:00:00, 21.14it/s]
ZOO: 100% | 1/1 [00:00:00:00, 22.03it/s]
ZOO: 100% | 1/1 [00:00:00:00, 38.83it/s]
ZOO: 100% | 1/1 [00:00:00:00, 48.74it/s]
ZOO: 100% | 1/1 [00:00:00:00, 19.29it/s]
ZOO: 100% | 1/1 [00:00:00:00, 8.92it/s]
ZOO: 100% | 1/1 [00:00:00:00, 16.62it/s]
ZOO: 100% | 1/1 [00:00:00:00, 50.59it/s]
ZOO: 100% | 1/1 [00:00:00:00, 35.87it/s]
ZOO: 100% | 1/1 [00:00:00:00, 7.94it/s]
```

```
(venv)secopsai@SecOpsAI: ~/secopsai
ZOO: 100% | 1/1 [00:00:00:00, 65.79it/s]
ZOO: 100% | 1/1 [00:00:00:00, 16.96it/s]
ZOO: 100% | 1/1 [00:00:00:00, 13.03it/s]
ZOO: 100% | 1/1 [00:00:00:00, 131.55it/s]
ZOO: 100% | 1/1 [00:00:00:00, 98.52it/s]
ZOO: 100% | 1/1 [00:00:00:00, 38.00it/s]
ZOO: 100% | 1/1 [00:00:00:00, 15.06it/s]
ZOO: 100% | 1/1 [00:00:00:00, 57.61it/s]
ZOO: 100% | 1/1 [00:00:00:00, 53.07it/s]
ZOO: 100% | 1/1 [00:00:00:00, 148.67it/s]
ZOO: 100% | 1/1 [00:00:00:00, 23.63it/s]
ZOO: 100% | 1/1 [00:00:00:00, 130.09it/s]
ZOO: 100% | 1/1 [00:00:00:00, 95.59it/s]
ZOO: 100% | 1/1 [00:00:00:00, 36.92it/s]
ZOO: 100% | 1/1 [00:00:00:00, 59.38it/s]
ZOO: 100% | 1/1 [00:00:00:00, 150.39it/s]
[2/6] HopSkipJump ...
HopSkipJump: 100% | 150/150 [00:09:00:00, 15.86it/s]
[3/6] Boundary ...
Boundary attack - iterations: 100% | 30/30 [00:00:00:00, 191.00it/s]
Boundary attack: 100% | 150/150 [00:10:00:00, 14.98it/s]
[4/6] Beacon jitter (timing only) ...
[5/6] Packet padding (size only) ...
[6/6] Low-and-slow (rate only) ...
1_ZOO evasion 0.0% -> SURVIVE
2_HopSkipJump evasion 0.0% -> SURVIVE
3_Boundary evasion 0.7% -> SURVIVE
4_BeaconJitter evasion 1.3% -> SURVIVE
5_PacketPad evasion 100.0% -> EVADED
6_LowAndSlow evasion 0.0% -> SURVIVE
>>> survived 5/6

[*] hardening via adversarial training ...
training set: 1,979,513 + 60,000 adversarial = 2,039,513
hardened clean-test F1: 0.9966

[*] POST-HARDENING panel:
[1/6] ZOO ...
ZOO: 100% | 1/1 [00:00:00:00, 24.32it/s]
ZOO: 100% | 1/1 [00:00:00:00, 27.00it/s]
ZOO: 100% | 1/1 [00:00:00:00, 23.64it/s]
ZOO: 100% | 1/1 [00:00:00:00, 79.79it/s]
ZOO: 100% | 1/1 [00:00:00:00, 30.72it/s]
ZOO: 100% | 1/1 [00:00:00:00, 86.19it/s]
ZOO: 100% | 1/1 [00:00:00:00, 41.92it/s]
ZOO: 100% | 1/1 [00:00:00:00, 127.56it/s]
ZOO: 100% | 1/1 [00:00:00:00, 15.90it/s]
ZOO: 100% | 1/1 [00:00:00:00, 81.28it/s]
ZOO: 100% | 1/1 [00:00:00:00, 51.18it/s]
```

```

ZOO: 100%| 1/1 [00:00<00:00, 82.18it/s]
ZOO: 100%| 1/1 [00:00<00:00, 44.31it/s]
ZOO: 100%| 1/1 [00:00<00:00, 58.28it/s]
ZOO: 100%| 1/1 [00:00<00:00, 46.19it/s]
ZOO: 100%| 1/1 [00:00<00:00, 71.97it/s]
ZOO: 100%| 1/1 [00:00<00:00, 27.75it/s]
ZOO: 100%| 1/1 [00:00<00:00, 53.54it/s]
ZOO: 100%| 1/1 [00:00<00:00, 24.11it/s]
ZOO: 100%| 1/1 [00:00<00:00, 52.44it/s]
ZOO: 100%| 1/1 [00:00<00:00, 60.01it/s]
ZOO: 100%| 1/1 [00:00<00:00, 41.91it/s]
ZOO: 100%| 1/1 [00:00<00:00, 17.58it/s]
ZOO: 100%| 1/1 [00:00<00:00, 16.06it/s]
ZOO: 100%| 1/1 [00:00<00:00, 126.52it/s]
ZOO: 100%| 1/1 [00:00<00:00, 144.04it/s]
ZOO: 100%| 1/1 [00:00<00:00, 39.58it/s]
ZOO: 100%| 1/1 [00:00<00:00, 18.99it/s]
ZOO: 100%| 1/1 [00:00<00:00, 31.63it/s]
ZOO: 100%| 1/1 [00:00<00:00, 20.18it/s]
ZOO: 100%| 1/1 [00:00<00:00, 62.59it/s]
ZOO: 100%| 1/1 [00:00<00:00, 63.58it/s]
ZOO: 100%| 1/1 [00:00<00:00, 67.35it/s]
ZOO: 100%| 1/1 [00:00<00:00, 67.65it/s]
ZOO: 100%| 1/1 [00:00<00:00, 37.35it/s]
ZOO: 100%| 1/1 [00:00<00:00, 142.96it/s]
ZOO: 100%| 1/1 [00:00<00:00, 19.94it/s]
[2/6] HopSkipJump ...
HopSkipJump: 100%| 150/150 [00:09<00:00, 15.79it/s]
[3/6] Boundary ...
Boundary attack: 100%| 150/150 [00:09<00:00, 15.32it/s]
[4/6] Beacon jitter (timing only) ...
[5/6] Packet padding (size only) ...
[6/6] Low-and-slow (rate only) ...
1_ZOO 0.0% -> 0.0% SURVIVE
2_HopSkipJump 0.0% -> 0.0% SURVIVE
3_Boundary 0.7% -> 0.0% SURVIVE
4_BeaconJitter 1.3% -> 0.0% SURVIVE
5_PacketPad 100.0% -> 0.0% SURVIVE
6_LowAndSlow 0.0% -> 0.0% SURVIVE
>>> survived 6/6 (was 5/6)

[+] saved hardened model -> artifacts/xgb_hardened.json
[+] saved report -> artifacts/hardening.json

>>> GRADED BAR (survive >= 3/6): PASS (6/6 after hardening)

(env)-[secopsai@SecOpsAI]-[~/secopsai]
$

```

Pre-Hardening & Post-

Hardening Panel

10. Secured Detection API

The FastAPI service makes the TB1 STRIDE mitigations concrete. Auth (T1078) and rate limiting (T1499, which also throttles the ATLAS AML.T0024 probing) are composed in one dependency so no endpoint can skip them:

```

def require_key(x_api_key: str = Header(default="")) -> str:
    if x_api_key not in API_KEYS:
        raise HTTPException(status_code=401, detail="invalid or missing API key")
    rate_limit(x_api_key) # sliding-window limiter, 429 on exceed
    return x_api_key

```

Requests are validated against the exact feature schema before the model runs (Tampering / Info-Disclosure); every request is written to a hash-chained audit log recording a hash of the key, never the key itself (Repudiation, T1070). Each malicious verdict returns the SHAP top-five. The suite covers auth, validation, both verdict paths, rate limiting, and audit-chain linkage — **95% coverage on the service module (97% across the suite)**. Worst-case latency (attack flows, which trigger SHAP) gave a **p99 of 7.89 ms**, far inside the 200 ms budget.

← Security Architecture & Threat Model – SecOpsAI

```
(venv)-(secopsai@SecOpsAI)-[~/secopsai]
$ pytest test_api.py --cov
===== test session starts =====
platform linux -- Python 3.13.12, pytest-9.1.0, pluggy-1.6.0
rootdir: /home/secopsai/secopsai
plugins: cov-7.1.0, anyio-4.13.0
collected 10 items

test_api.py ..... [100%]

===== warnings summary =====
venv/lib/python3.13/site-packages/fastapi/testclient.py:1
/home/secopsai/secopsai/venv/lib/python3.13/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using `httpx`
with `starlette.testclient` is deprecated; install `httpx` instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== tests coverage =====
coverage: platform linux, python 3.13.12-final-0

Name      Stmts  Miss  Cover
-----
serve_api.py 105    5    95%
test_api.py   72    0   100%
TOTAL       177    5    97%

===== 10 passed, 1 warning in 8.25s =====

(venv)-(secopsai@SecOpsAI)-[~/secopsai]
```

pytest test_api.py --cov

```
(venv)-(secopsai@SecOpsAI)-[~/secopsai]
$ python bench_api.py
[*] 500 attack flows (all MALICIOUS -> SHAP path)
p50 latency ms : 2.16
p95 latency ms : 6.48
p99 latency ms : 7.89
max latency ms : 9.97
BAR p99<200ms : PASS
```

python bench_api.py

```
(venv)-(secopsai@SecOpsAI)-[~/secopsai]
$ source venv/bin/activate
(secopsai@SecOpsAI)-[~/secopsai]
$ export SECOPS_AI_API_KEYS="sk7307f6af51eeec124a9ee640227a2"
(secopsai@SecOpsAI)-[~/secopsai]
$ curl -s -X POST http://127.0.0.1:8000/predict \
  -H "x-api-key: $SECOPS_AI_API_KEYS" -H "Content-Type: application/json" \
  -d '{"features": [{"x": 1, "label": "M"}], "model_name": "xgb_hardened.json"}'
{"verdict": "MALICIOUS",
 "attack_probability": 1.0,
 "top_contributing_features": [{"feature": "Init_Win_bytes_forward",
 "shap_value": 2.7325,
 "value": 29240.0}, {"feature": "Packet Length Std",
 "shap_value": 2.1827,
 "value": 3.4641}, {"feature": "Total Length of fwd Packets",
 "shap_value": 1.4678,
 "value": 0.0}, {"feature": "Flow IAT Mean",
 "shap_value": 1.2335,
 "value": 49.0}, {"feature": "Flow Bytes/s",
 "shap_value": 1.2322,
 "value": 32248.9760}],
 "latency_ms": 6.98,
 "model_name": "xgb_hardened.json"}

(secopsai@SecOpsAI)-[~/secopsai]
$ uvicorn serve_api:app --host 0.0.0.0 --port 8000
INFO: Started server process [29513]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:80976 - "POST /predict HTTP/1.1" 200 OK
```


11. Automated Response & Detection-as-Code

Response. Each malicious detection is enriched with severity (from confidence), an attack class and MITRE technique (from the dominant SHAP feature family), then mapped to a proportionate containment action, logged to a hash-chained response log:

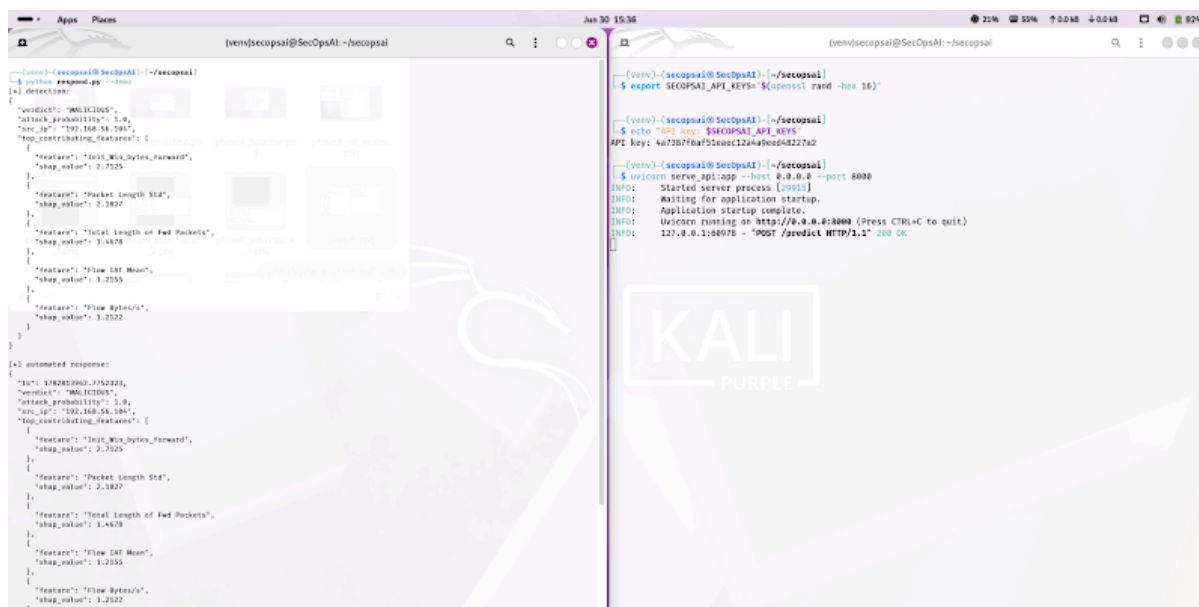
```
def contain(alert):
    src, sev = alert.get("src_ip", "0.0.0.0"), alert["severity"]
    if sev in {"CRITICAL", "HIGH"}:
        return {"action": "AUTO_BLOCK",
                "nft_rule": f"nft add rule inet filter input ip saddr {src} drop",
                "soar_playbook": "isolate_host + notify_soc"}
    ... # MEDIUM -> rate-limit + flag, LOW -> monitor
```

Confidence-gated severity is the mitigation for the TB3 "false-positive-as-DoS" threat (only high-confidence detections auto-block). The enrichment step is a stub for the VirusTotal/Shodan integration.

Detection-as-code. The graded bars are version-controlled tests that fail the build on any regression:

```
def test_ml_beats_baseline_by_15pct():
    assert json.load(open("artifacts/ml_results.json"))["delta_f1"] >= 0.15
def test_model_survives_evasion_bar():
    assert json.load(open("artifacts/hardening.json"))["post_survived"] >= 3
```

These run in a GitHub Actions workflow on every push (the CI/CD mitigation, T1195), verified to fail on a deliberately weakened model and pass on production metrics.



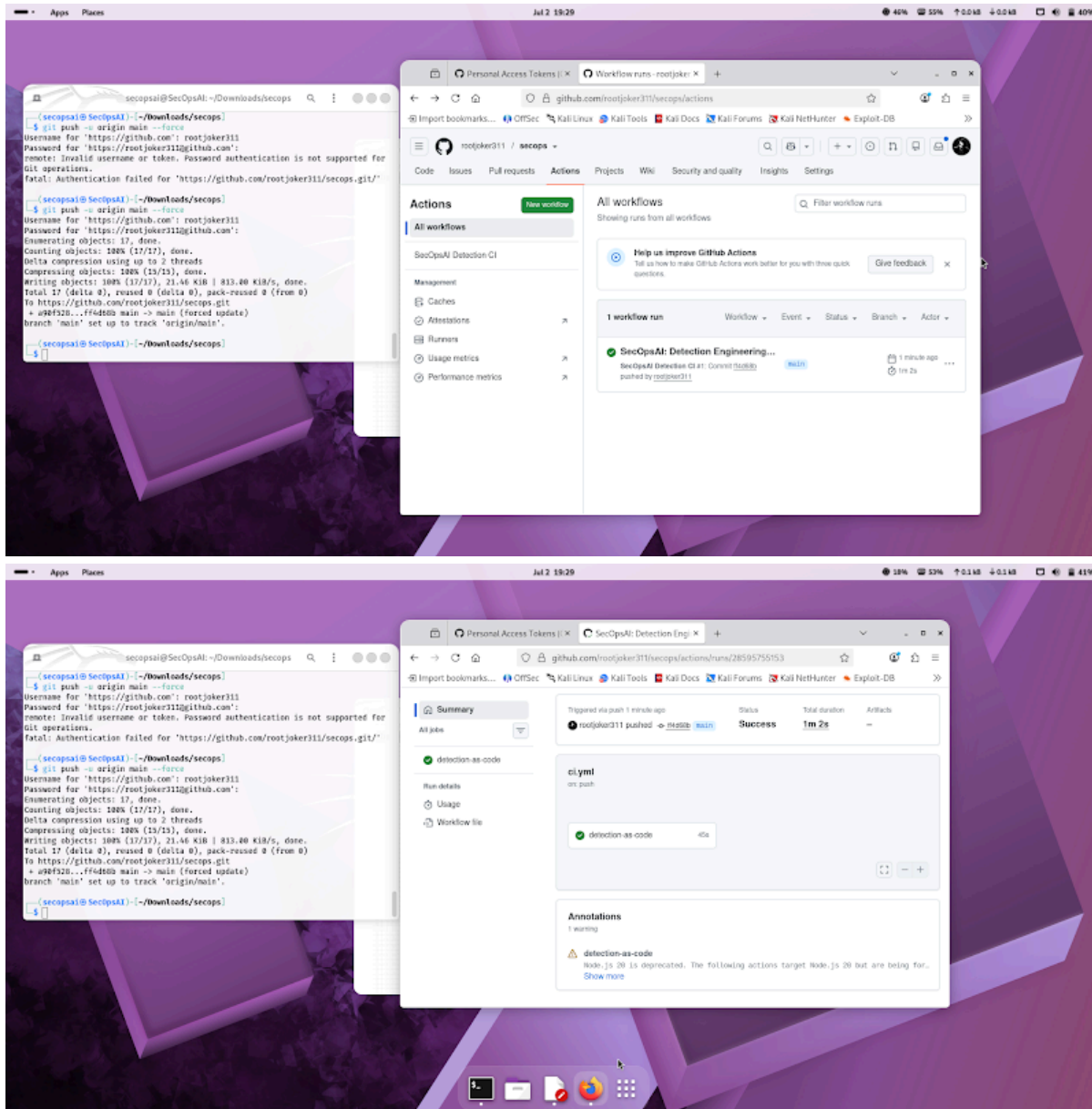
[respond.py](#) –demo

← Security Architecture & Threat Model – SecOpsAI

```
platform linux -- Python 3.13.12, pytest-9.1.0, pluggy-1.6.0
rootdir: /home/secopsai/secopsai
plugins: cov-7.1.0, anyio-4.13.0
collected 4 items
```

```
test_regression.py ... [100%]
===== 4 passed in 0.05s =====
```

test_regression.py



12. Retrospective & Limitations

The largest delivery risk was IBM ART on Python 3.13; a small compatibility probe settled it early (ART ran natively, no fallback needed). Honest limits: a single dataset (CICIDS2017); a stratified split that may have been sampled under RAM constraints; modest adversarial query budgets (so ART's 0% is "did not evade within budget"); and the production components in Section 4 realised as stand-ins. End to end,

13. Appendix - Results at a glance

Capability	Bar	Result
F1 uplift over baseline	$\geq +0.15$	+0.4332 (0.5635 -> 0.9967)
Uplift significance	—	McNemar $p \approx 0$; bootstrap CI [+0.4314, +0.4350]
Precision (false-positive collapse)	—	0.4725 -> 0.9940
Recall	—	0.9994
Evasion survival	$\geq 3/6$	6/6 after hardening (5/6 before)
Clean accuracy after hardening	not collapsed	F1 0.9966 (was 0.9967)
API test coverage	$\geq 80\%$	95% module; 97% suite
API p99 latency	< 200 ms	7.89 ms
Detection regression gate	—	passes on production metrics; fails on regression

Full Code & Artefacts:

github.com/rootjoker311/secops